ATMOS AI

Color & Weather Intelligence

Deep Learning Semester Project Report

Project	ATMOS AI — Color & Weather Intelligence
Model runtime	TensorFlow.js LayersModel
Input	224 × 224 RGB image
Status	Technical package prepared; some submission details require verification

Abstract

ATMOS AI is a browser-based computer-vision demonstration that analyzes an image or live camera frame and presents estimates for dominant color, brightness, color temperature, and weather-related scene categories. The supplied application loads a TensorFlow.js LayersModel in the browser and performs inference locally after the model has loaded. The interface includes image upload, live camera input, camera switching, a compact analysis dashboard, and visual color-composition bars.

The supplied artifacts confirm a 224 × 224 × 3 model input and four prediction outputs: 11 color categories, 3 brightness categories, 3 temperature categories, and 6 weather categories. The application also applies image-pixel heuristics for night/sky cues. These heuristics are not a substitute for a separately validated weather detector.

1. Problem Statement

Interpreting visual scenes can require several related cues at once: color distribution, illumination, color tone, and possible weather conditions. The project provides a single interface for demonstrating these cues from a photo or camera frame. The goal is educational and demonstrative: combine a multi-output image model with a responsive browser interface and make the predictions understandable to a user.

Scope: this project estimates visual properties from an image. It does not measure meteorological variables, does not replace a weather station or forecast service, and should not be treated as a safety-critical weather warning system.

2. Objectives

1. Load a trained TensorFlow.js model in a browser without requiring an inference server.

2. Accept either an uploaded image or a live camera stream.

3. Produce separate predictions for color, brightness, temperature-tone, and weather categories.

4. Display model outputs and a simple visual summary in a responsive interface.

5. Package source code, model JSON, binary weight shards, documentation, and presentation resources for reproducibility.

3. Dataset Description

The supplied class metadata lists 11 color labels (red, orange, yellow, green, cyan, blue, purple, pink, black, white, gray), 3 brightness labels (dark, medium, bright), 3 temperature-tone labels (cool, neutral, warm), and 6 weather labels (cloudy, foggy, other, rainy, shine, sunrise). These labels describe the output taxonomy, not the underlying dataset source or sample counts.

The uploaded files do not include a verified dataset URL, dataset license, class-wise image counts, split sizes, or the original training/validation manifest. Those details must be added from the actual dataset used by the project. Do not claim that the data came from a particular public dataset until the exact source is confirmed.

Dataset URL: PENDING — insert the verified URL for the exact dataset used to train this ATMOS AI model.

Before final submission, record the source, license/usage terms, total image count, counts per class/task, training/validation/test split strategy, and any preprocessing or augmentation applied.

4. Model Architecture

The exported model is a TensorFlow.js LayersModel. The model configuration contains a MobileNetV2 feature extractor with a 224 × 224 × 3 input and a shared dense representation of 256 units with ReLU activation. Four softmax heads provide task-specific outputs: color_pct (11), brightness (3), temperature (3), and weather (6). The JSON and three binary weight shards are included in the deployment package.

Preprocessing in the web application resizes each frame to 224 × 224, converts pixels to float values, and scales them to approximately [-1, 1] before calling the model. This is what the current client code implements; it should be checked against the exact preprocessing used during training before formal benchmarking.

5. System Workflow

1. User selects an image or grants camera access.

2. The browser loads the TensorFlow.js runtime and model JSON plus weight shards.

3. The source frame is resized and normalized in the browser.

4. The model returns four prediction vectors.

5. The application displays the highest-scoring color, brightness, temperature-tone, and weather-related output.

6. A pixel-based heuristic checks dark-pixel ratios and simple upper-frame sky-like cues to help decide whether to present a weather label or color-only summary.

7. The dashboard displays color bars, inference time, and camera-motion estimate where available.

6. Technology Stack

Frontend: HTML, CSS, JavaScript. Model runtime: TensorFlow.js 4.22.0 loaded from a CDN. Model format: TensorFlow.js LayersModel with model.json and binary weight shards. Input: image file or browser camera via getUserMedia. Deployment target requested by the course: Vercel static hosting.

7. Evaluation and Results

Functional capability visible in the supplied code: model loading, image upload, camera input, multi-output inference, color visualization, latency display, and heuristic weather-related presentation. The model files are present and the output dimensions are consistent with the supplied class metadata.

No verified held-out test metrics (accuracy, precision, recall, F1-score, per-class confusion matrix), sample counts, or training logs were supplied with the uploaded browser model. Therefore this report does not invent an accuracy percentage. Before final submission, run an evaluation on a held-out dataset and include measured metrics for each output head, as well as example predictions and failure cases.

8. Limitations

Predictions depend on the quality and distribution of the training data. The source and label provenance need to be documented. The weather logic includes hand-written pixel heuristics and can misclassify indoor scenes, low-light images, or unusual skies. Temperature here is a visual tone category, not physical temperature in degrees. The application runs client-side, so initial model download and inference speed depend on device/browser performance and network availability.

9. Privacy and Deployment

The code describes inference as browser-side after model load. The application does not intentionally upload the selected image to an inference server. Browser camera permission is required for live input, and camera access generally requires HTTPS or localhost. For public deployment, ensure the repository includes index.html, the tfjs_model directory, and assets referenced by the page. Test the deployed site on desktop and mobile after every release.

10. Reproducibility and Submission Checklist

• Public GitHub repository with index.html, model.json, all referenced weight shards, class metadata, report, presentation, and a README.

• Exact dataset source URL and license/usage details added; document image counts and splits.

• Verified Vercel URL added to the website and README.

• Report link opens from the deployed site.

• Presentation and narration script are accessible.

• Record the presentation voiceover, upload to YouTube as Public or Unlisted, and add the URL.

• Publish a LinkedIn project post and add its URL to the final submission form.

• Run a held-out evaluation and include real metrics; do not use guessed numbers.

• Test the model assets and camera controls from a clean browser session.

References and Project Links

Dataset source: PENDING — insert the verified URL for the exact dataset used to train this ATMOS AI model.

GitHub repository: PENDING — insert the public GitHub repository URL.

Vercel website: PENDING — insert the deployed Vercel URL.

YouTube presentation: PENDING — insert the public/unlisted recorded presentation URL.

LinkedIn post: PENDING — insert the LinkedIn post URL.

Note: replace each PENDING entry with the actual link before submission.